
Table of Contents

.....	1
Question 1:	1
Question 2	2
Question 3	2
Question 4	2
Question 5	3
Question 5.1 and 6	3
Question 7	4
Question 8	4
Question 9	4
Question 10	5
Question 11	5
Question 12	5
Question 13	6
Question 14	8
Question 15	8
Question 16	8
Question 17	8
Question 18	10
Question 19	12

```
clc  
clear
```

Question 1:

average, removes details and captures the measured trend removes unwanted frequencies from a signal above a determined cutoff frequency

```
% so in other words it is setting each pixel of the output image as the  
% average of the original pixel values and its neighborhood; and the average  
% can be a weighted average  
% by adjusting the weight or parameters of the filter.
```

```
img0 = imread('Lena.bmp');  
figure(1), imshow(img0)
```



Question 2

```
filt_avg1 = 1/9*ones(3);  
img1_LP = uint8(filter2( filt_avg1, img0 ));
```

Question 3

```
filt_avg2 = ones(3,3)/16; filt_avg2(2,2) = 0.5;  
img1_LP2 = uint8(filter2( filt_avg2, img0 ));
```

Question 4

```
subplot(2,2,1), imshow(img0), title('original')  
subplot(2,2,2), imshow(img1_LP), title('averaging filter 1')  
subplot(2,2,3), imshow(img1_LP2), title('averaging filter 2')
```



Question 5

The difference between the original image and its smoothened version reveals where the original image is busy i.e having different pixel values around edges and textured areas

Question 5.1 and 6

The third line of code, seems to factor in the grey scale values to the sharpening filter

```
% The sharpening filter seems to outline the edge features of the face such  
% as eyes, hair, her hat etc.
```

```
img0_HP = double(img0) - double(img1_LP);
```

```
figure(2), subplot(1, 2, 1), imshow(img0), title('Original'); axis equal  
subplot(1, 2, 2), imshow(img0_HP, []), title('High-pass Filter1'), colorbar
```

```
SP_factor = 1; % parameter to control the degree of sharpening  
temp = double(img0) + double(img0_HP) * SP_factor;  
img0_SP = uint8( (temp > 255) * 255 + (temp >= 0 & temp <= 255) .* temp );  
% clip the values outside [0, 255] range and convert to uint8
```

```
SP_factor2 = 2;  
temp = double(img0) + double(img0_HP) * SP_factor2;  
img0_SP2 = uint8( (temp > 255) * 255 + (temp >= 0 & temp <= 255) .* temp );
```

```
subplot(1, 3, 1), imshow(img0_SP), title('Sharpened')
```

```
% Question 6
subplot(1, 3, 2), imshow(img0_SP2), title('Sharpened 2')
```



Question 7

% The averaging filter would attempt smoothen out the noisy pixels

Question 8

The sharpening filter would take the busy pixels and make them pop out more since it may see the busy location as edges

Question 9

% Filters 1 and 2 detect edges in the horizontal and vertical directions

% The first filter is tuned to take the difference between surrounding left and right pixels

% The second filter is tuned to take the difference between surrounding pixels above and below each image pixel being processed

```
edge_filt1_1 = [-1, 0, 1; -1, 0, 1; -1, 0, 1], % "Prewitt" edge filters
edge_filt1_2 = [-1, -1, -1; 0, 0, 0; 1, 1, 1],
```

```
img1_edge1_1 = filter2( edge_filt1_1 / 3, img0 ); % apply edge filters
img1_edge1_2 = filter2( edge_filt1_2 / 3, img0 );
```

```
edge_filt1_1 =
```

```

-1      0      1
-1      0      1
-1      0      1
```

```
edge_filt1_2 =
```

```

-1      -1      -1
 0       0       0
 1       1       1
```

Question 10

%pixel positions where an abrupt change of intensity in greyscale (or color) values occur

Question 11

I assume it means its taking the derivative(gradient?) of positions to find the change at a particular pixel value

Question 12

edge filter 2 has a cleaner, more sharper image than edge filter 1

```
img1_edge1_2(1,:) = 0; % handle cases on image border img1_edge1_1(:, 1) = 0;
img1_edge1_2(size(img0,1),:) = 0; img1_edge1_1(:, size(img0,2)) = 0;
```

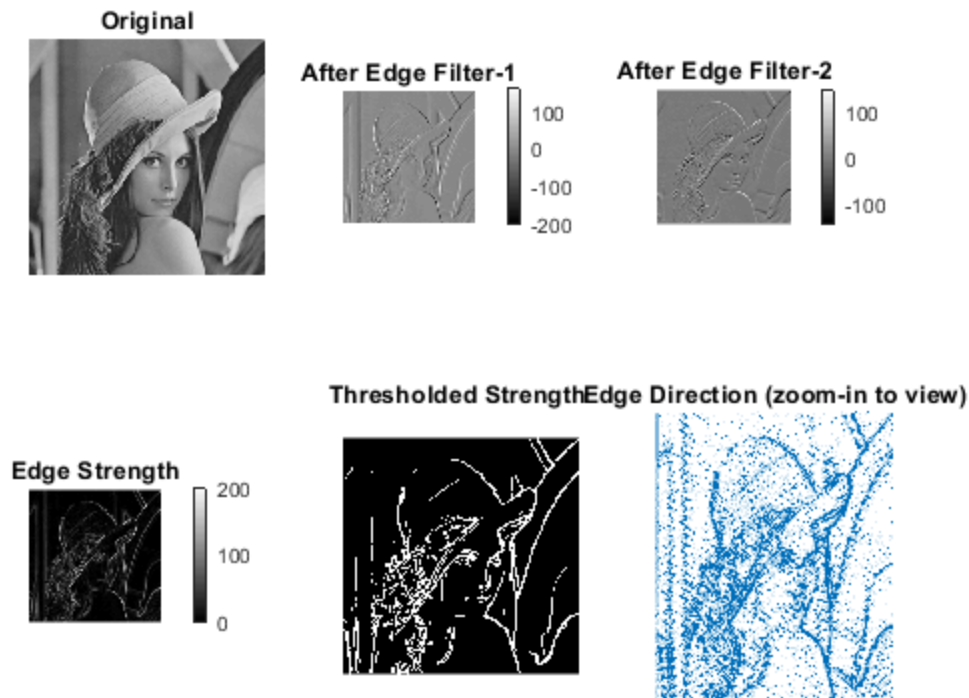
```
img1_edge1 = sqrt( double(img1_edge1_1) .^2 + double(img1_edge1_2) .^2 );
edge_th = 40; % threshold of edge strength
```

```
figure(4), subplot(2, 3, 1), imshow(img0), title('Original')
subplot(2, 3, 2), imshow(img1_edge1_1, []), title('After Edge Filter-1'),
    colorbar
subplot(2, 3, 3), imshow(img1_edge1_2, []), title('After Edge Filter-2'),
    colorbar
subplot(2, 3, 4), imshow(img1_edge1, []), title('Edge Strength'), colorbar
subplot(2, 3, 5), imshow(img1_edge1 > edge_th, []), title('Thresholded
    Strength ')
axis equal
subplot(2, 3, 6), st=1; %% visualize edge directions at a selected resolution
    quiver( img1_edge1_2( size(img1,1):(-st):1, 1:st:size(img1,2) ), ...
    quiver( img1_edge1_2( size(img0,1):(-st):1, 1:st:size(img0,2) ), ...
```

```

img1_edge1_1( size(img0,1):(-st):1, 1:st:size(img0,2) ) ),
axis([1, size(img0,2), 1, size(img0,1) ]), axis off
title('Edge Direction (zoom-in to view)')

```



Question 13

Looks like a gradient vector field(dont have enough math knowledge on this subject) the arrows are pointing in which direction the pixel values are changing?

```

figure(5); st = 1;
quiver( img1_edge1_2( size(img0,1):(-st):1, 1:st:size(img0,2) ), ...
img1_edge1_1( size(img0,1):(-st):1, 1:st:size(img0,2) ) ),
axis([1, size(img0,2), 1, size(img0,1) ]), axis off, title('Edge Direction
(zoom-in to view)')

img1_edge3 = edge(img0, 'sobel');
img1_edge4 = edge(img0, 'canny');
figure(6), subplot(1, 3, 1), imshow(img0), title('Original')
subplot(1, 3, 2), imshow(img1_edge3), title('Sobel Edge Detector')
subplot(1, 3, 3), imshow(img1_edge4), title('Canny Edge Detector')

```

Edge Direction (zoom-in to view)



Original



Sobel Edge Detector



Canny Edge Detector



Question 14

```
openExample('images/DetectCirclesExample')
```

```
% It finds circular objects that are brighter than the background to find the  
lighter circles
```

Question 15

it would not work. To find circle we would need to use the new tools given in the matlab example which requires us to draw the radii of the circle before it uses an algorithm to identify them

Question 16

A directional change in the intensity or color in an image.

Question 17

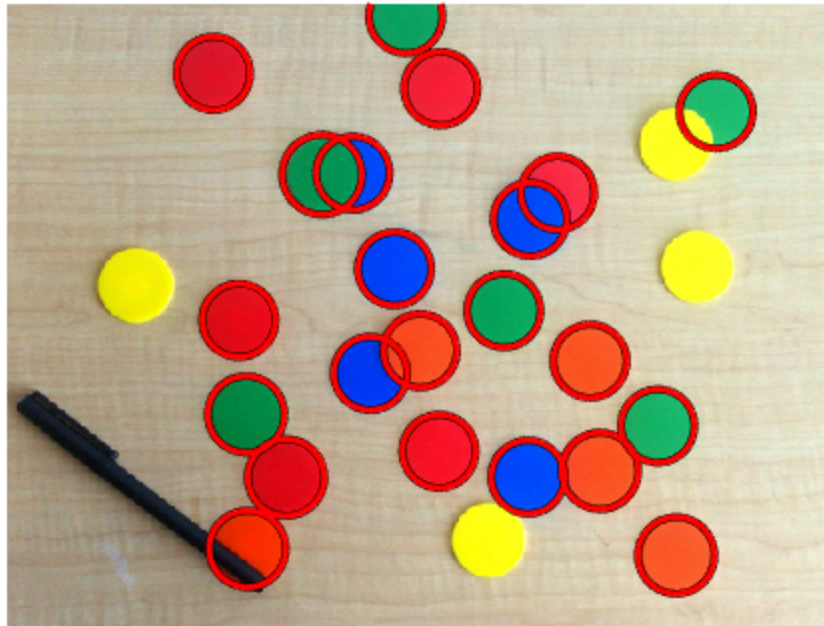
```
rgb = imread('coloredChips.png');  
figure(10), imshow(rgb);  
d = drawline;  
  
pos = d.Position;  
diffPos = diff(pos);  
diameter = hypot(diffPos(1), diffPos(2))  
  
gray_image = rgb2gray(rgb);  
imshow(gray_image);  
  
[centers, radii] = imfindcircles(rgb, [20 25], 'ObjectPolarity', 'dark');  
[centers, radii] = imfindcircles(rgb, [20 25], 'ObjectPolarity', 'dark', ...  
    'Sensitivity', 0.9);  
  
[centers, radii] = imfindcircles(rgb, [20 25], 'ObjectPolarity', 'dark', ...  
    'Sensitivity', 0.92);  
imshow(rgb)  
h = viscircles(centers, radii);  
  
[centers, radii] = imfindcircles(rgb, [20 25], 'ObjectPolarity', 'dark', ...  
    'Sensitivity', 0.92);  
  
length(centers)  
  
delete(h) % Delete previously drawn circles  
h = viscircles(centers, radii);  
  
[centers, radii] = imfindcircles(rgb, [20 25], 'ObjectPolarity', 'dark', ...  
    'Sensitivity', 0.92, 'Method', 'twostage');
```

```
delete(h)
h = viscircles(centers,radii);
```

```
diameter =  
        6691/178
```

```
ans =  
    16
```





Question 18

```
[centers,radii] = imfindcircles(rgb,[20 25], 'ObjectPolarity','dark', ...
    'Sensitivity',0.95);
[centersBright,radiiBright,metricBright] = imfindcircles(rgb,[20 25], ...
    'ObjectPolarity','bright','Sensitivity',0.92,'EdgeThreshold',0.1);

delete(h)
viscircles(centers,radii);

imshow(gray_image);

[centersBright,radiiBright] = imfindcircles(rgb,[20 25], ...
    'ObjectPolarity','bright','Sensitivity',0.92);

imshow(rgb);

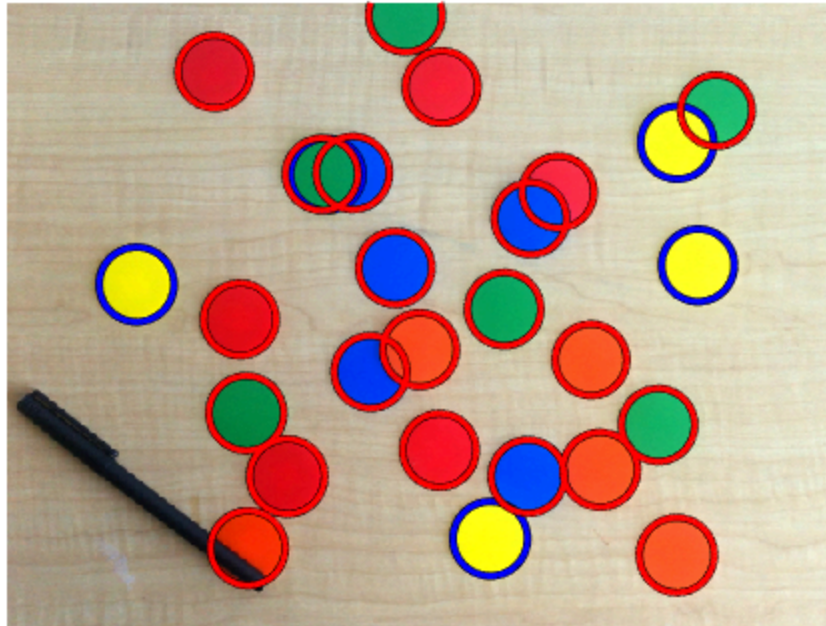
hBright = viscircles(centersBright, radiiBright, 'Color','b');

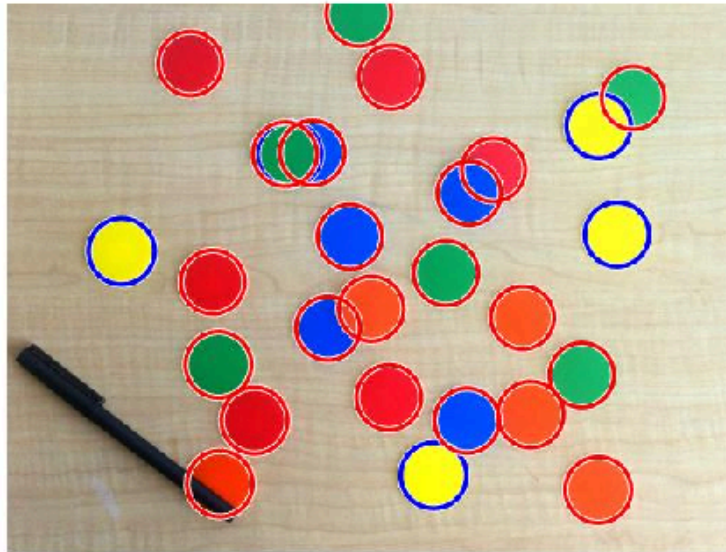
[centersBright,radiiBright,metricBright] = imfindcircles(rgb,[20 25], ...
    'ObjectPolarity','bright','Sensitivity',0.92,'EdgeThreshold',0.1);

delete(hBright)
hBright = viscircles(centersBright, radiiBright, 'Color','b');

h = viscircles(centers,radii);
```

```
img00 = imread('img777.jpg');  
figure(7), imshow(img00)
```





Question 19

the stitching isn't perfect. you can see at parts where the image don't line up properly, You can clearly see that the images aren't connected to also the black background makes it seem unprofessional

```
% Load images.
buildingDir = fullfile(toolboxdir('vision'),'visiondata','tahoe');
buildingScene = imageDatastore(buildingDir);

% Display images to be stitched.
montage(buildingScene.Files)

% Read the first image from the image set.
I = readimage(buildingScene,1);

% Initialize features for I(1)
grayImage = im2gray(I);
points = detectSURFFeatures(grayImage);
[features, points] = extractFeatures(grayImage,points);

% Initialize all the transforms to the identity matrix. Note that the
% projective transform is used here because the building images are fairly
% close to the camera. Had the scene been captured from a further distance,
% an affine transform would suffice.
numImages = numel(buildingScene.Files);
```

```

tforms(numImages) = projective2d(eye(3));

% Initialize variable to hold image sizes.
imageSize = zeros(numImages,2);

% Iterate over remaining image pairs
for n = 2:numImages

    % Store points and features for I(n-1).
    pointsPrevious = points;
    featuresPrevious = features;

    % Read I(n).
    I = readimage(buildingScene, n);

    % Convert image to grayscale.
    grayImage = im2gray(I);

    % Save image size.
    imageSize(n,:) = size(grayImage);

    % Detect and extract SURF features for I(n).
    points = detectSURFFeatures(grayImage);
    [features, points] = extractFeatures(grayImage, points);

    % Find correspondences between I(n) and I(n-1).
    indexPairs = matchFeatures(features, featuresPrevious, 'Unique', true);

    matchedPoints = points(indexPairs(:,1), :);
    matchedPointsPrev = pointsPrevious(indexPairs(:,2), :);

    % Estimate the transformation between I(n) and I(n-1).
    tforms(n) = estimateGeometricTransform2D(matchedPoints,
    matchedPointsPrev,...
        'projective', 'Confidence', 99.9, 'MaxNumTrials', 2000);

    % Compute T(n) * T(n-1) * ... * T(1)
    tforms(n).T = tforms(n).T * tforms(n-1).T;
end

% Compute the output limits for each transform.
for i = 1:numel(tforms)
    [xlim(i,:), ylim(i,:)] = outputLimits(tforms(i), [1 imageSize(i,2)], [1
    imageSize(i,1)]);
end

avgXLim = mean(xlim, 2);
[~,idx] = sort(avgXLim);
centerIdx = floor((numel(tforms)+1)/2);
centerImageIdx = idx(centerIdx);

Tinv = invert(tforms(centerImageIdx));
for i = 1:numel(tforms)
    tforms(i).T = tforms(i).T * Tinv.T;

```

```

end

for i = 1:numel(tforms)
    [xlim(i,:), ylim(i,:)] = outputLimits(tforms(i), [1 imageSize(i,2)], [1
        imageSize(i,1)]);
end

maxImageSize = max(imageSize);

% Find the minimum and maximum output limits.
xMin = min([1; xlim(:)]);
xMax = max([maxImageSize(2); xlim(:)]);

yMin = min([1; ylim(:)]);
yMax = max([maxImageSize(1); ylim(:)]);

% Width and height of panorama.
width = round(xMax - xMin);
height = round(yMax - yMin);

% Initialize the "empty" panorama.
panorama = zeros([height width 3], 'like', I);

blender = vision.AlphaBlender('Operation', 'Binary mask', ...
    'MaskSource', 'Input port');

% Create a 2-D spatial reference object defining the size of the panorama.
xLimits = [xMin xMax];
yLimits = [yMin yMax];
panoramaView = imref2d([height width], xLimits, yLimits);

% Create the panorama.
for i = 1:numImages

    I = readimage(buildingScene, i);

    % Transform I into the panorama.
    warpedImage = imwarp(I, tforms(i), 'OutputView', panoramaView);

    % Generate a binary mask.
    mask = imwarp(true(size(I,1),size(I,2)), tforms(i), 'OutputView',
        panoramaView);

    % Overlay the warpedImage onto the panorama.
    panorama = step(blender, panorama, warpedImage, mask);
end

figure(12)
imshow(panorama)

```



Published with MATLAB® R2021b